CAMPUS CAFÉ ORDERING SYSTEM

Name : Patenio, Angeline B.
Maninang, Rona Mae C.
Section : SEC D
Date: 11-10-2026
Subject : Object-Oriented Programming

I.INTRODUCTION

The Campus Café Ordering System is a simple Java program that stimulates how customers order food and beverages in a café. It demonstrates the use of Object Oriented Programing (OOP) concepts, including classes,objects, inheritance, encapsulation, abstraction, and polymorphism. The system also calculates item prices, applies student discounts and vouchers, and awards loyalty points**.**

II. OOP CONCEPT EXPLANATION

1.Classes and Objects

Classes serve as blueprints for creating objects. In this program, classes such as Beverage, Pastry, Meal, and Customer are used to create different café items and customers.

2.Encapsulation

Encapsulation is used by declaring variables as private or protected and accessing them through getters and setters. This protects the data and allows validation, such as preventing negative prices and loyalty points.

3.Inheritance

Inheritance is demonstrated when `Beverage`, `Pastry`, and `Meal` extend the abstract `MenuItem` class. This allows the subclasses to reuse common attributes and methods.

4.Abstraction

Abstraction is implemented through the abstract `MenuItem` class and its abstract methods `computePrice()` and `getDescription()`. It provides a common structure while allowing each item type to have its own implementation.

5.Polymorphism

Polymorphism is demonstrated when different menu item objects are stored using the `MenuItem` type. Each object uses its own version of `computePrice()` and `getDescription()`.

6.Method Overloading

Method overloading is used in the `Customer` class through `addPoints(int)` and `addPoints(double)`. The `Order` class also has two `computeTotal()` methods, one with a voucher parameter and one without.

7.Method Overriding

Method overriding is used when the subclasses provide their own implementations of `computePrice()` and `getDescription()`. Each menu item calculates its price based on its own additional features.

8.Constructors

Constructors are used to initialize objects. Default and parameterized constructors are provided for the menu item classes, while the `Customer` class has different constructors for creating customers with different information.

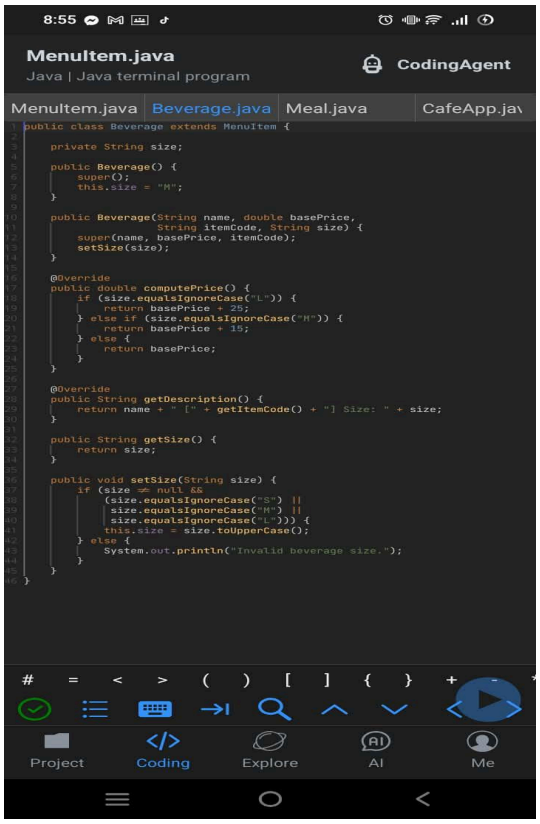
9.Conditional Statements

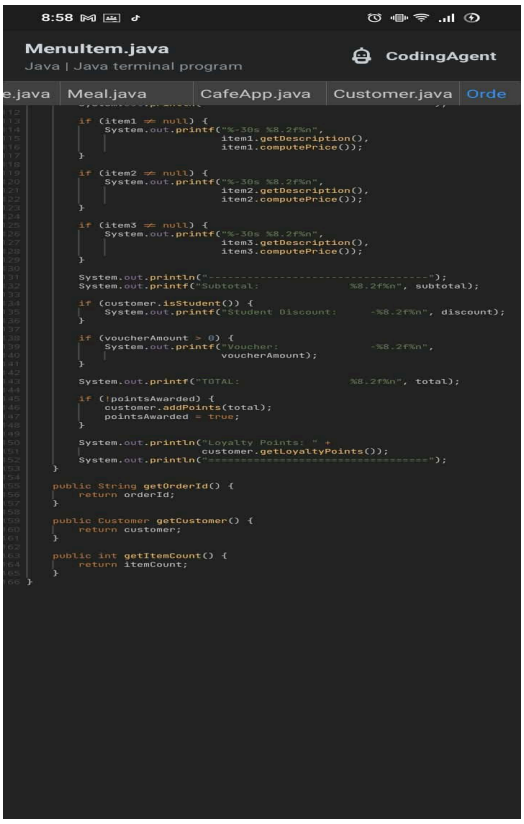
Conditional statements such as `if`, `else if`, and `else` are used to check student discounts, item limits, valid prices, beverage sizes, and additional food options.

III.PROGRAM OUTPUT

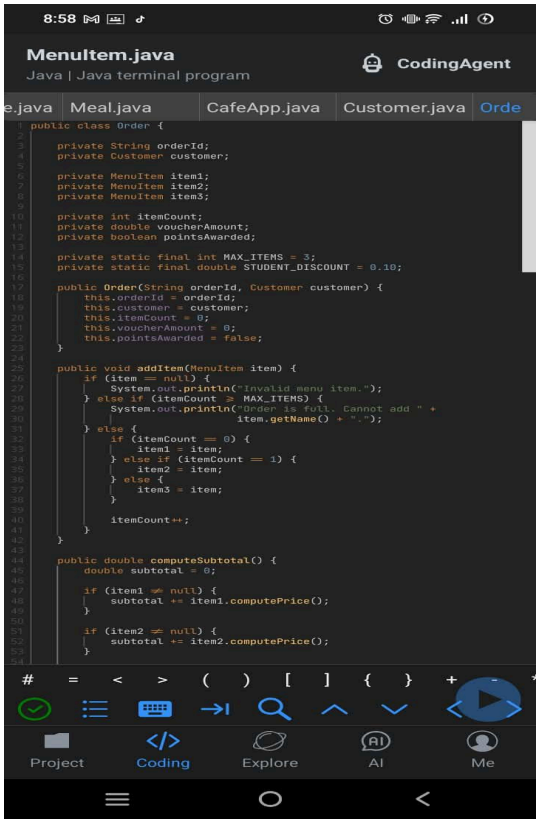





8:55
MenuItem.java
Java | Java terminal program
CodingAgent
MenuItem.java
Beverage.java
Meal.java
CafeApp.jav
public class Beverage extends MenuItem {
private String size;
public Beverage() {
super();
this.size = "M";
}
public Beverage(String name, double basePrice,
String itemCode, String size) {
super(name, basePrice, itemCode);
setSize(size);
}
@Override
public double computePrice() {
if (size.equalsIgnoreCase("L")) {
return basePrice + 25;
} else if (size.equalsIgnoreCase("M")) {
return basePrice + 15;
} else {
return basePrice;
}
}
@Override
public String getDescription() {
return name + " [" + getItemCode() + "] Size: " + size;
}
public String getSize() {
return size;
}
public void setSize(String size) {
if (size != null &&
(size.equalsIgnoreCase("S") ||
size.equalsIgnoreCase("M") ||
size.equalsIgnoreCase("L"))) {
this.size = size.toUpperCase();
} else {
System.out.println("Invalid beverage size.");
}
}
}
= < > ( ) [ ] { } + -
Project
Coding
Explore
AI
Me

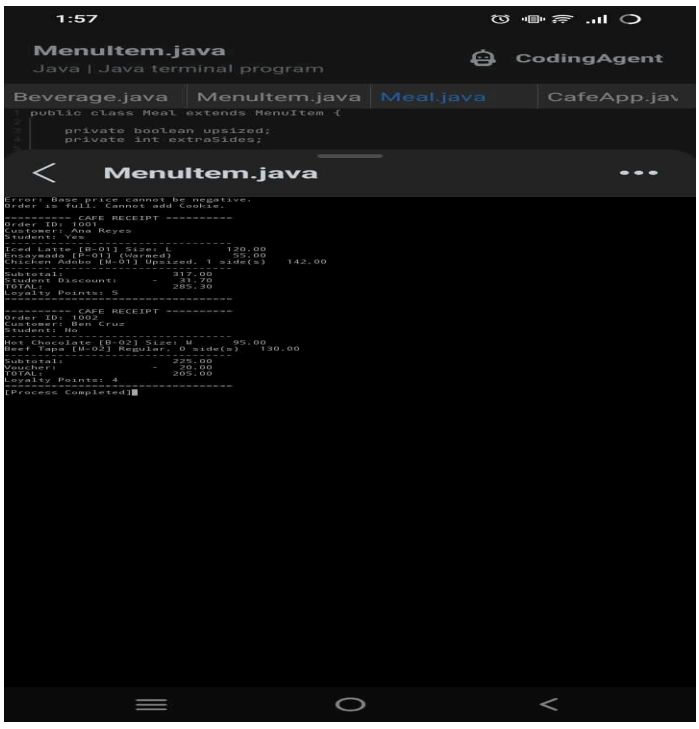

8:58
MenuItem.java
Java | Java terminal program
CodingAgent
Meal.java
CafeApp.java
Customer.java
Orde
if (item1 != null) {
System.out.printf("%-30s %8.2f%n",
item1.getDescription(),
item1.computePrice());
}
if (item2 != null) {
System.out.printf("%-30s %8.2f%n",
item2.getDescription(),
item2.computePrice());
}
if (item3 != null) {
System.out.printf("%-30s %8.2f%n",
item3.getDescription(),
item3.computePrice());
}
System.out.println("----------");
System.out.printf("Subtotal: %8.2f%n", subtotal);
if (customer.isStudent()) {
System.out.printf("Student Discount: -%8.2f%n", discount);
}
if (voucherAmount > 0) {
System.out.printf("Voucher: -%8.2f%n",
voucherAmount);
}
System.out.printf("TOTAL: %8.2f%n", total);
if (!pointsAwarded) {
customer.addPoints(total);
pointsAwarded = true;
}
System.out.println("Loyalty Points: " +
customer.getLoyaltyPoints());
System.out.println("==========");
}
public String getOrderId() {
return orderId;
}
public Customer getCustomer() {
return customer;
}
public int getItemCount() {
return itemCount;
}
}


8:57
MenuItem.java
Java | Java terminal program
CodingAgent
age.java
Meal.java
CafeApp.java
Customer.java
Order.jav
public class Customer {
private String name;
private boolean student;
private int loyaltyPoints;
public Customer() {
this("Guest", false);
}
public Customer(String name) {
this(name, false);
}
public Customer(String name, boolean student) {
this.name = name;
this.student = student;
this.loyaltyPoints = 0;
}
public void addPoints(int points) {
if (points > 0) {
loyaltyPoints += points;
}
}
public void addPoints(double amount) {
if (amount > 0) {
loyaltyPoints += (int) (amount / 50);
}
}
public String getName() {
return name;
}
public void setName(String name) {
if (name != null && !name.trim().isEmpty()) {
this.name = name;
}
}
public boolean isStudent() {
return student;
}
public void setStudent(boolean student) {
this.student = student;
}
public int getLoyaltyPoints() {
return loyaltyPoints;
}
public void setLoyaltyPoints(int loyaltyPoints) {
if (loyaltyPoints >= 0) {
this.loyaltyPoints = loyaltyPoints;
} else {
System.out.println("Error: Loyalty points cannot be negative.");
}
}
}


8:58
MenuItem.java
Java | Java terminal program
CodingAgent
e.java
Meal.java
CafeApp.java
Customer.java
Orde
public class Order {
private String orderId;
private Customer customer;
private MenuItem item1;
private MenuItem item2;
private MenuItem item3;
private int itemCount;
private double voucherAmount;
private boolean pointsAwarded;
private static final int MAX_ITEMS = 3;
private static final double STUDENT_DISCOUNT = 0.10;
public Order(String orderId, Customer customer) {
this.orderId = orderId;
this.customer = customer;
this.itemCount = 0;
this.voucherAmount = 0;
this.pointsAwarded = false;
}
public void addItem(MenuItem item) {
if (item == null) {
System.out.println("Invalid manu item.");
} else if (itemCount >= MAX_ITEMS) {
System.out.println("Order is full. Cannot add " +
item.getName() + ".");
} else {
if (itemCount == 0) {
item1 = item;
} else if (itemCount == 1) {
item2 = item;
} else {
item3 = item;
}
itemCount++;
}
}
public double computeSubtotal() {
double subtotal = 0;
if (item1 != null) {
subtotal += item1.computePrice();
}
if (item2 != null) {
subtotal += item2.computePrice();
}
= < > ( ) [ ] { } + -
Project
Coding
Explore
AI
Me


8:57
MenuItem.java
Java | Java terminal program
CodingAgent
MenuItem.java
Beverage.java
Meal.java
CafeApp.java
C
public class CafeApp {
public static void main(String[] args) {
// Create customers using different constructors.
Customer customer1 = new Customer("Ana Reyes", true);
Customer customer2 = new Customer("Ben Cruz");
// Create beverage objects.
Beverage latte = new Beverage(
"Iced Latte", 95.00, "B-01", "L"
);
Beverage hotChocolate = new Beverage();
hotChocolate.setName("Hot Chocolate");
hotChocolate.setBasePrice(-5);
hotChocolate.setBasePrice(80.00);
hotChocolate.setItemCode("B-02");
hotChocolate.setSize("M");
// Create pastry objects.
Pastry ensaymada = new Pastry(
;
Pastry cookie = new Pastry();
cookie.setName("Cookie");
cookie.setBasePrice(30.00);
cookie.setItemCode("P-02");
cookie.setWarmed(false);
// Create meal objects.
Meal chickenAdobo = new Meal(
"Chicken Adobo", 110.00, "M-01", true, 1
);
Meal beefTapa = new Meal();
beefTapa.setName("Beef Tapa");
beefTapa.setBasePrice(130.00);
beefTapa.setItemCode("M-02");
beefTapa.setExtraSides(0);
// First order.
Order order1 = new Order("1001", customer1);
order1.addItem(latte);
order1.addItem(ensaymada);
order1.addItem(chickenAdobo);
// Test the maximum of three items.
order1.addItem(cookie);
order1.printReceipt();
// Second order.
Order order2 = new Order("1002", customer2);
order2.addItem(hotChocolate);
order2.addItem(beefTapa);
// Apply a 20-peso voucher.
order2.computeTotal(20.00);
order2.printReceipt();
}
}


8:58
MenuItem.java
Java | Java terminal program
CodingAgent
Meal.java
CafeApp.java
Customer.java
Orde
if (item3 != null) {
subtotal += item3.computePrice();
}
return subtotal;
}
public double computeTotal() {
double subtotal = computeSubtotal();
if (customer.isStudent()) {
return subtotal - (subtotal * STUDENT_DISCOUNT);
} else {
return subtotal;
}
}
public double computeTotal(double voucher) {
if (voucher < 0) {
System.out.println("Invalid voucher amount.");
voucher = 0;
}
double total = computeTotal();
if (voucher > total) {
voucher = total;
}
voucherAmount = voucher;
return total - voucherAmount;
}
public void printReceipt() {
double subtotal = computeSubtotal();
double discount = 0;
if (customer.isStudent()) {
discount = subtotal * STUDENT_DISCOUNT;
}
double discountedTotal = subtotal - discount;
double total = discountedTotal - voucherAmount;
if (total < 0) {
total = 0;
}
System.out.println();
System.out.println("========== CAFE RECEIPT ==========");
System.out.println("Order ID: " + orderId);
System.out.println("Customer: " + customer.getName());
System.out.println("Student: " +
= < > ( ) [ ] { } + -
Project
Coding
Explore
AI
Me


1:57
MenuItem.java
Java | Java terminal program
CodingAgent
Beverage.java
MenuItem.java
Meal.java
CafeApp.jav
public class Meal extends MenuItem {
private boolean upsized;
private int extraSides;
MenuItem.java
Error: Base price cannot be negative.
Order is full. Cannot add Cookie.
========== CAFE RECEIPT ==========
Order ID: 1001
Customer: Ana Reyes
Student: Yes
Iced Latte [B-01] Size: L 120.00
Ensaymada [P-01] [Warmed] 55.00
Chicken Adobo [M-01] Upsized, 1 side(s) 142.00
Subtotal: 317.00
Student Discount: - 31.70
TOTAL: 285.30
Loyalty Points: 5
========== CAFE RECEIPT ==========
Order ID: 1002
Customer: Ben Cruz
Student: No
Hot Chocolate [B-02] Size: M 95.00
Beef Tapa [M-02] Regular, 0 side(s) 130.00
Subtotal: 225.00
Voucher: - 20.00
TOTAL: 205.00
Loyalty Points: 4
[Process Completed]

IV. CHALLENGES ENCOUNTERED AND SOLUTIONS

One challenge we encountered was understanding how inheritance and method overriding work together. I solved this by using `MenuItem` as the parent class and allowing its subclasses to implement their own price calculations.

Another challenge was making sure that invalid values, such as negative prices and extra sides, would not be accepted. I solved this by adding validation inside the setter methods.

we also had to make sure that the order could only accept three items. I used conditional statements to check the item count before adding another item.

V. CONCLUSION

Through this activity, we learned how to apply the basic concepts of Object-Oriented Programming in Java. we also understood how classes, objects, inheritance, encapsulation, abstraction, and polymorphism can work together to create a simple but functional ordering system. This exercise improved our understanding of Java programming and problem-solving.